

MCP 调用链卡顿问题调试复盘

从怀疑 Tavily 到定位 LangSmith 追踪

适用项目 agent-dev-lab

复盘日期 2026 年 9 月 15 日

核心结论

Tavily API、Tavily Key、search_jobs 函数和 FastMCP 调度均能独立完成。卡顿只在 MCP stdio 子进程启用 LangSmith 自动追踪时出现。该子进程的 LangSmith 请求经过本机 Clash Verge Mihomo 代理端口 127.0.0.1:7897 后没有及时完成，使 web_search.ainvoke 一直等待追踪相关回调。

本文记录完整排查过程，并给出在保留 LangSmith 追踪的前提下检查 endpoint、代理规则、工作区配置和 MCP 子进程环境的方法。

摘要与建议

这次问题最容易误判的原因，是日志最后停在 `search_jobs` 内部、`Tavily` 调用之前。表面现象指向 `Tavily`，但控制变量实验显示 `Tavily` 本身能够正常返回，真正的差异变量是 `LangSmith tracing` 是否启用。

已经确认的事实

- `TAVILY_API_KEY` 能从 `.env` 读取，非空、无首尾空格，格式符合 `tvly` 前缀。
- 同一虚拟环境在正常网络权限下调用 `Tavily`，API 响应时间约 0.88 秒并返回 5 条结果。
- 直接调用 `search_jobs` 成功；直接通过 `FastMCP` 工具调度器调用也成功。
- 通过 `MCP stdio` 子进程调用且 `tracing` 为 `true` 时，程序进入 `search_jobs` 后持续等待。
- 把 `LANGSMITH_TRACING=false` 明确传给 `MCP` 子进程后，完整 `MCP` 调用约 4.27 秒完成。
- 卡住的 `MCP` 子进程连接到了 `127.0.0.1:7897`，该端口由 `verge-mihomo.exe` 监听。

保持追踪时的处理顺序

1. 确认 `LangSmith` 账号所在区域，并设置正确的 `LANGSMITH_ENDPOINT`。
2. 让 `Clash Verge Mihomo` 对 `LangSmith` 域名使用可用代理节点，不要命中 `REJECT` 或异常的 `DIRECT` 规则。
3. 如 API Key 关联多个 `workspace`，设置 `LANGSMITH_WORKSPACE_ID`。
4. 在 `MCP connection` 的 `env` 字段中显式传递 `LangSmith` 配置，不依赖子进程碰巧找到项目根目录下的 `.env`。
5. 按 `LangSmith` 连通性测试、`search_jobs` 单测、`MCP` 工具测试、`main.py` 的顺序逐层回归。

结论的边界

实验已经证明 `tracing` 为 `true` 是 `MCP` 卡顿的必要差异条件，并证明关闭 `tracing` 后链路恢复。当前证据还不能把更底层原因唯一归为某条 `Clash` 规则、某个代理节点、`LangSmith` 区域 `endpoint` 或账号 `workspace` 配置。后续应使用有超时的 `LangSmith Client` 测试继续区分这些可能性。

系统调用链

先把程序拆成可独立验证的层，才能知道一次卡顿究竟属于业务函数、第三方 API、框架包装还是进程通信。

```
main.py
-> run_career_agent
  -> LangGraph agent
    -> MCP client
      -> stdio 启动 Python 子进程
        -> career_server.py
          -> search_jobs
            -> TavilySearch.ainvoke
              -> api.tavily.com/search
                -> api.smith.langchain.com 追踪上报
```

TavilySearch.ainvoke 不只是一次 HTTP 搜索。当全局 tracing 开启时，LangChain 还会创建 trace、记录输入和输出，并由 LangSmith 客户端上传。因此，ainvoke 停住并不能单凭最后一条业务日志判断是 Tavily 请求停住。

涉及的项目位置

位置	作用	本次观察
main.py	启动完整 Agent	完整链路变量最多，不适合作为第一层定位工具
framework_agent/agent.py	创建 Agent 并调用模型和工具	模型、RAG、MCP、追踪都会混在一次运行中
mcp_clients/career_client.py	通过 stdio 创建 MCP 子进程	子进程环境由 MCP SDK 重新组装
mcp_servers/career_server.py	注册 search_jobs 并调用 Tavily	日志停在第 43 行附近
.env	保存 API Key 和 tracing 配置	LANGSMITH_TRACING=true

为什么完整 main.py 不适合一开始就反复运行

完整 Agent 同时依赖大模型接口、MCP 子进程、Tavily、向量库和 LangSmith。任何一层失败都可能覆盖下一层的真实状态。本次受限环境运行 main.py 时，大模型连接先报 OpenAIConnectionError，程序甚至没有机会进入 search_jobs。这不是 Tavily 的证据，只说明完整测试无法隔离变量。

初始现象和假设

已知现象是 search_jobs 被调用后程序报错或长时间没有返回。最初不能直接认定 Tavily 故障，需要同时保留多种解释。

假设	如果成立应看到什么	验证方法
Tavily Key 缺失或格式错误	对象初始化失败，或服务返回 401 403	只检查是否存在、长度和前缀，不打印密钥
Tavily 参数写错	ainvoke 本地校验报 ValidationError	查看工具签名并直接调用
Tavily 服务不可达	无法建立 api.tavily.com 的 HTTPS 连接	隔离 Tavily 并在正常网络权限下执行
MCP 包装层出错	直接函数成功，MCP 调用失败	建立有无 MCP 的对照组
追踪系统阻塞	tracing true 卡住，false 恢复	只改变 tracing 一个变量
终端编码问题	结果已返回，但打印含特殊字符时报 UnicodeEncodeError	区分计算完成与输出失败

第一原则 先确定卡在哪一层

面对长调用链，调试不是从最复杂的入口反复尝试，而是逐层建立最小可运行实验。每次实验只回答一个问题，并保留一个对照组。

第二原则 区分事实和解释

日志显示 search_jobs entered 是事实。把它解释为 Tavily 挂了仍然只是一个假设。只有把 Tavily 单独调用成功、MCP 调用失败、关闭 tracing 后恢复这三组证据放在一起，才足以把故障范围收敛到追踪链路。

逐步调试记录

步骤一 阅读调用链和配置

先查看 main.py、Agent、MCP client、MCP server、Tavily 测试文件以及 .env 的变量名。没有打印任何 Key 内容，只检查变量是否存在、字符串长度、首尾空格和已知前缀。

```
检查结果
TAVILY_API_KEY: 已加载, 长度 57, 以 tvly 开头
LANGSMITH_TRACING: true
LANGSMITH_PROJECT: agent-dev-lab
```

这一结果排除了 Key 完全没加载，但不能证明 Key 有效。只有 Tavily 服务器返回成功结果才能验证 Key。

步骤二 检查 Tavily 的异常处理方式

检查已安装的 langchain-tavily 源码后发现，TavilySearch 的异步实现会捕获普通异常并返回一个 error 字典，而不是继续抛出异常。career_server.py 又执行 return str(result)。因此 MCP 可能把网络失败当成普通字符串返回，错误位置会被掩盖。

```
except Exception as e:
    return {"error": e}

# 项目代码随后执行
return str(result)
```

这个设计不是本次卡顿的根因，但它解释了为什么日志和错误表现容易令人误判。

步骤三 受限环境中的第一次复现

最初在受限诊断环境中直接运行 Tavily 测试，得到 ClientConnectorError 和 PermissionError 13。LangSmith 也同时连接失败。这个结果只能说明当前诊断容器不允许外网连接，不能直接代表用户电脑。

```
ClientConnectorError(host='api.tavily.com', port=443,
PermissionError(13, '拒绝访问'))
```

这里出现了本次调试中的一次错误方向：把受限执行环境的网络策略当成了项目真实故障。

步骤四 使用浏览器证据纠正判断

浏览器访问 api.tavily.com 返回 JSON message Alive，证明 Tavily 域名和浏览器网络链路可达。这仍不能验证 Python 的 POST 请求和 API Key，但足以否定电脑完全无法访问 Tavily 的说法。于是需要在用户明确授权后，用正常网络权限执行隔离测试。

步骤五 在正常网络权限下隔离 Tavily

运行项目中已有的 test_tavily.py，并临时在当前进程关闭 LangSmith tracing，确保测试只回答 Tavily 是否正常。

```
$env:LANGSMITH_TRACING='false'
python -m agent_dev_lab.test_tavily
```

观测项	结果
进程耗时	约 3.09 秒
Tavily API response_time	0.88 秒
结果数量	5 条
API Key	认证成功
结论	Tavily API、Key、查询参数和异步调用方式均正常

步骤六 运行完整 MCP 工具链

随后运行已有的 test_career_client.py。MCP 能完成 ListToolsRequest，也能发出 CallToolRequest，服务端打印 search_jobs entered，但超过 60 秒没有返回结果。

```
Processing request of type ListToolsRequest
Processing request of type CallToolRequest
1. search_jobs entered
# 此后持续等待
```

这一步把问题范围从 Tavily 缩小为 MCP 路径中的某个差异，但还不能判断是 FastMCP 调度、stdio 通信、子进程环境还是 tracing。

步骤七 直接调用同一个 search_jobs

跳过 MCP client 和 stdio 子进程，直接导入并运行 career_server.py 中的 search_jobs。相同中文查询约 3.82 秒成功返回。

这说明 search_jobs 函数、全局 TavilySearch 对象、中文查询和 async await 写法本身没有问题。故障只在进入 MCP 子进程路径后出现。

步骤八 直接调用 FastMCP 调度器

为了区分 FastMCP 调度器和 stdio 子进程，再通过 `mcp.call_tool` 直接调用已注册工具。约 2.47 秒成功返回。

至此可以排除 FastMCP 的工具注册和调度逻辑，剩余重点是 stdio 子进程环境及其网络副作用。

步骤九 观察卡住进程的网络状态

在 MCP 测试卡住时，读取 Python 进程树和 TCP 连接。子进程没有直接显示到 Tavily 的公网连接，而是连接到本机 127.0.0.1:7897。该端口的监听进程是 verge-mihomo.exe，即 Clash Verge Mihomo 的代理核心。

观测	含义
MCP server 是 client 启动的子进程	stdio 进程模型符合预期
子进程连接 127.0.0.1:7897	请求经过本机代理
连接保持 Established 或进入 CloseWait	连接建立过，但响应或关闭流程未正常完成
直接 Tavily 调用成功	代理并非对所有域名和所有 Python 请求都不可用

步骤十 检查 MCP 子进程环境继承

MCP SDK 的 stdio_client 不会完整继承父进程环境，而是只传递 APPDATA、PATH、TEMP、USERPROFILE 等白名单变量。父进程临时设置的 LANGSMITH_TRACING=false 不在白名单中。

```
DEFAULT_INHERITED_ENV_VARS = [  
    'APPDATA', 'HOMEDRIVE', 'HOMEPATH', 'LOCALAPPDATA',  
    'PATH', 'PATHEXT', 'PROCESSOR_ARCHITECTURE', 'SYSTEMDRIVE',  
    'SYSTEMROOT', 'TEMP', 'USERNAME', 'USERPROFILE'  
]
```

子进程启动后，career_server.py 又调用 load_dotenv，因此它重新从项目 .env 读取 LANGSMITH_TRACING=true。于是父进程看似关闭追踪，MCP 子进程实际上仍然启用了追踪。

步骤十一 只改变 tracing 变量

最后建立关键对照组：其他代码、Key、查询、MCP transport 和代理都保持不变，只在 MCP connection 的 env 中明确传入 LANGSMITH_TRACING=false。

实验	Tracing	结果	耗时
MCP stdio 原配置	true	进入 search_jobs 后持续等待	> 60 秒
MCP stdio 对照配置	false	工具返回搜索结果	约 4.27 秒

对照测试返回结果后，诊断命令在向 GBK 终端打印特殊字符时出现 UnicodeEncodeError。这个错误发生在结果已经返回之后，属于诊断输出编码问题，不影响 tracing 对照实验的结论。

推理如何收敛

问题	实验	结论
Key 是否缺失	只读检查 .env	否，Key 已加载
Key 是否有效	正常网络下请求 Tavily	是，返回 5 条结果
query 参数是否错误	直接 ainvoke	否
search_jobs 是否错误	直接调用同一函数	否
FastMCP 调度是否错误	直接 mcp.call_tool	否
stdio 路径是否相关	MCP client 完整测试	是，只在该路径卡住
tracing 是否关键变量	仅切换 tracing true false	是，false 时恢复
Tavily 是否是最终根因	综合全部对照	否

确认结论

MCP stdio 子进程启用 LangSmith tracing 时，search_jobs 的 LangChain 调用无法及时完成。关闭 tracing 后，同一调用链恢复。Tavily 只是被追踪的那次运行，不是失败的第三方服务。

最可能的底层机制

LangSmith 会在 LangChain 调用前后记录 run，并访问 api.smith.langchain.com 获取服务信息和上传 trace。卡住进程的连接经过 Clash Verge Mihomo。结合此前出现的 LangSmithConnectionError，最可能的机制是 LangSmith 域名对应的代理路由、代理节点、TLS 链路或区域 endpoint 没有正常完成，而 tracing 回调延长了 ainvoke 的完成时间。

尚未完全证明的部分

- 具体是哪一条 Clash 规则或哪个代理节点导致 LangSmith 请求停住。
- LangSmith 账号属于 US、APAC、EU 还是 AWS US 区域。
- API Key 是否关联多个 workspace，是否必须设置 LANGSMITH_WORKSPACE_ID。

- 直接调用 LangSmith Client.info 时会返回成功、认证错误还是网络超时。

保留这些不确定性很重要。好的调试报告应说明证据能证明什么，也要说明还没有证明什么。

保留 LangSmith 追踪的修改建议

目标不是关闭追踪，而是让 MCP 子进程稳定、明确地连接到正确的 LangSmith endpoint，并让网络失败能够快速暴露。建议按以下顺序处理。

第一步 确认 LangSmith 区域

LangSmith 官方文档说明，默认 endpoint 是 US Cloud。账号如果位于 APAC 或 EU，必须使用对应区域地址，否则 Key 可能无法正确认证。

```
# US Cloud
LANGSMITH_ENDPOINT=https://api.smith.langchain.com

# APAC
LANGSMITH_ENDPOINT=https://apac.api.smith.langchain.com

# EU
LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com
```

在 LangSmith 控制台确认账号区域，只保留其中一个 endpoint。不要因为人在中国就直接假定账号一定属于 APAC；以账号实际部署区域为准。

第二步 检查 Clash Verge Mihomo 规则

在 Clash 连接日志中搜索 smith.langchain.com。确认它没有命中 REJECT，也不要使用无法访问该域名的异常节点。可以先用一条范围较窄的规则进行验证，代理组名称需要替换成自己的实际名称。

```
DOMAIN-SUFFIX,smith.langchain.com,PROXY
```

修改规则后，先在浏览器访问 <https://api.smith.langchain.com/info>。能快速返回 JSON 或明确的 401 403，都比一直加载更有价值：前者说明服务可达，后者说明网络可达但认证或区域配置还需处理。

第三步 完善 .env

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=<现有 LangSmith Key>
LANGSMITH_PROJECT=agent-dev-lab
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
# 仅当 Key 可访问多个 workspace 时添加
# LANGSMITH_WORKSPACE_ID=<Workspace ID>
```

不要把 Key 写进 Python 文件，也不要再在日志中打印 Key。官方文档指出，多 workspace Key 才需要 LANGSMITH_WORKSPACE_ID。

第四步 显式传递 MCP 子进程环境

虽然当前子进程能通过工作目录找到 .env，但显式传递配置更可靠，也允许父进程覆盖某个变量进行诊断。以下是建议结构，Key 仍从环境变量读取。

```
import os
from dotenv import load_dotenv

load_dotenv()

langsmith_env = {
    name: os.getenv(name)
    for name in (
        'LANGSMITH_TRACING',
        'LANGSMITH_API_KEY',
        'LANGSMITH_PROJECT',
        'LANGSMITH_ENDPOINT',
        'LANGSMITH_WORKSPACE_ID',
    )
    if os.getenv(name)
}

# 在 career.connection 内增加
'env': langsmith_env,
```

这项修改解决的是子进程配置可预测性，不能替代 endpoint 和代理连通性修复。如果 LangSmith 域名仍无法访问，显式传入 tracing=true 仍然会卡住。

第五步 用短超时测试 LangSmith

先单独验证 LangSmith，不要直接运行 main.py。下面的诊断只请求服务信息，不运行 Agent。
timeout_ms 可以避免网络异常时等待很久。

```
from dotenv import load_dotenv
from langsmith import Client

load_dotenv()
client = Client(timeout_ms=(5000, 10000))
print(client.info)
```

如果该测试在 10 秒内返回信息，说明 endpoint、代理和基本客户端连接已正常。若快速返回 401 或 403，重点检查 Key、区域和 workspace；若连接超时，重点检查 Clash 规则和代理节点。

第六步 分层回归

- 6. 先运行 LangSmith Client.info，确认追踪服务可达。
- 7. 运行 test_tavily.py，确认搜索不受追踪影响。
- 8. 运行 test_career_client.py，确认 MCP 子进程能返回。
- 9. 最后运行 main.py，验证模型、RAG、MCP 和 tracing 的组合。
- 10. 到 LangSmith 控制台确认 agent-dev-lab 项目出现完整 trace。

建议的失败判读表

现象	优先检查
Client.info 超时	代理规则、节点、DNS、TLS、endpoint 可达性
Client.info 返回 401	API Key 和区域 endpoint
Client.info 返回 workspace 错误	LANGSMITH_WORKSPACE_ID
LangSmith 单测成功但 MCP 卡住	career connection 的 env 与 cwd
MCP 成功但 main.py 失败	模型接口、RAG 或 Agent 逻辑

可复用的调试方法

一 画出调用链

先写出入口、框架、子进程、业务函数、第三方服务和旁路服务。旁路服务包括日志、监控和 tracing，它们经常被忽略，却可能改变主调用的完成时间。

二 建立最小复现

从最靠近外部依赖的一层开始。本次先测试 Tavily，再测试 search_jobs，再测试 FastMCP，最后才测试 stdio MCP。每增加一层，如果结果从成功变为失败，新增的那一层就是重点。

三 一次只改变一个变量

最后的 tracing true false 对照最有证明力，因为其他条件没有变化。若同时更换 Key、代理、包版本和代码，就无法知道是哪项修改生效。

四 为每一层设置观测点

进入函数 -> 构造参数 -> 外部调用前 -> 外部调用后 -> 返回前
进程层面 -> PID 和父子关系 -> TCP 目标 -> 连接状态
配置层面 -> 变量是否存在 -> 来源 -> 是否传给子进程

观测点应该记录状态和耗时，不应打印密码、Token、完整简历或其他敏感内容。

五 对环境差异保持警惕

浏览器、IDE、普通终端、受限沙箱和 MCP 子进程可能使用不同的代理、证书、环境变量和工作目录。某个环境中失败，不足以推出另一个环境也失败。

六 主动寻找反证

看到 PermissionError 后，浏览器 Alive 页面是重要反证。好的调试不是不断收集支持原假设的证据，而是设计能够推翻原假设的实验。

七 记录结论边界

本次可以确认 tracing 是关键差异变量，但不能仅凭现有证据断言具体某条 Clash 规则错误。把已确认事实、最可能解释和未验证项分开，能够避免在修复时再次走偏。

常见误区

误区	为什么不可靠	更好的做法
最后一条日志在 Tavily 前，所以 Tavily 挂了	ainvoke 还包含 tracing 回调	拆开测试 Tavily 和 tracing
浏览器能打开，所以 Python 一定能访问	代理、证书和请求方式可能不同	用同一虚拟环境发出实际请求
Key 存在，所以 Key 一定有效	存在只证明加载，不证明服务接受	观察服务端成功或明确状态码
完整 main.py 报错就是目标模块报错	更早的模型调用可能先失败	从外部依赖开始逐层增加组件
关闭 tracing 能运行，所以问题解决了	这只是绕过，不满足保留追踪的目标	修复 endpoint 代理和子进程配置
一次修改很多配置更快	无法建立因果关系	每次只改变一个变量并记录耗时

通用排查清单

- ☐ 复现问题并记录完整异常链、最后日志和耗时。
- ☐ 画出调用链，标记进程边界和第三方服务。
- ☐ 检查配置是否存在，但不输出秘密值。
- ☐ 隔离最底层外部依赖并验证 Key 和网络。
- ☐ 逐层加入业务函数、框架包装和进程通信。
- ☐ 建立成功与失败的对照组，一次只改变一个变量。
- ☐ 卡顿时检查进程树、网络目标和连接状态。
- ☐ 检查父进程与子进程的环境变量、cwd、编码和代理差异。
- ☐ 把根因、伴随问题和诊断噪声分开。
- ☐ 修复后按从小到大的顺序回归，并确认监控端收到数据。

本次证据时间线

阶段	测试	结果	推理变化
代码阅读	检查 main 到 Tavily 的调用链	定位调用点和 tracing 配置	形成多假设列表
配置检查	只看 Key 状态和前缀	Tavily Key 已加载	排除完全缺失
受限环境	隔离 Tavily	PermissionError 13	产生网络怀疑，但证据受环境限制
浏览器反证	访问 api.tavily.com	返回 Alive	纠正电脑无网的过度结论
正常权限	test_tavily.py	5 条结果，API 0.88 秒	排除 Tavily API Key 参数
完整 MCP	test_career_client.py	进入函数后 > 60 秒等待	锁定 MCP 路径差异
直接函数	search_jobs	约 3.82 秒成功	排除函数实现
FastMCP 调度	mcp.call_tool	约 2.47 秒成功	排除工具注册和调度
进程观察	TCP 和监听进程	连接 verge-mihomo 7897	发现代理路径
环境检查	stdio 环境白名单	父进程覆盖不自动传递	发现子进程配置差异
最终对照	子进程 tracing=false	约 4.27 秒返回	确认 tracing 为关键差异变量

诊断过程中没有做的事情

- 没有修改项目源代码、.env 或 API Key。
- 没有把任何完整密钥写入终端输出或报告。
- 一次性 Python 对照代码通过 python -c 在进程内执行，没有保存到项目源码目录。
- 没有终止用户已经启动的 main.py 和 MCP 子进程。

参考资料

以下资料用于核对 LangSmith tracing、endpoint、workspace 和按运行时传递环境配置的官方要求。

1. [LangSmith 创建账号和 API Key](#)
2. [LangChain 应用启用 LangSmith tracing](#)
3. [不依赖环境变量的 tracing 配置](#)
4. [LangSmith profile 和 endpoint 优先级](#)
5. [沙箱运行时传递 tracing 环境](#)

最后的学习重点

这次调试最值得复用的能力，不是记住 LANGSMITH_TRACING 这个变量，而是把复杂系统拆成可以独立验证的层，并通过对照实验寻找唯一差异。看到一个错误时，先问它属于哪个环境、哪一层、由谁抛出、是否被包装，以及什么实验能够推翻当前猜测。